Week 8 — Drift and Observability Monitoring

Course: DS5619 Machine Learning Systems Operations | **Track B:** BMD-45 Vehicle Detection
Student Roll Number: **142301024** | **Deterministic Seed:** **706208327** | **Date:** October 2026

---

1. Executive Summary & Problem Context

In production machine learning systems, models degrade silently when real-world operational distributions drift away from training conditions. In the BMD-45 benchmark (Sharma et al., CVPR 2026 Findings), an object detector trained on out-of-domain data exhibited a catastrophic performance collapse—achieving only **33.6% mAP** on real-world CCTV footage compared to **83.8% mAP** when trained in-domain. Because ground-truth labels are rarely available at inference time, unsupervised observability monitoring of model outputs (e.g. prediction confidence scores) is vital to detect drift before severe failures impact downstream systems.

This submission operationalizes univariate feature drift monitoring using the **Population Stability Index (PSI)**. Personalized synthetic CCTV datasets were generated using roll number **142301024**, comparing a reference camera (camera_A_daylight) against a live traffic camera (camera_B_lowlight). The pipeline extracts continuous confidence scores, computes PSI, categorizes drift levels, and outputs a standardized drift report.

2. Mathematical Foundations: Population Stability Index (PSI)

The Population Stability Index quantifies how much a monitored ('live') distribution *P* has shifted from a baseline ('reference') distribution *Q* across *B* equal-width bins:

PSI Mathematical Formulation:

$$\mathbf{PSI = \sum_{b=1..B} (P_b - Q_b) \times \ln(P_b / Q_b)}$$

Symmetric Kullback-Leibler (Jeffrey) Divergence:

$$PSI = D_{KL}(P \,||\, Q) + D_{KL}(Q \,||\, P)$$
$$= \Sigma\, P_b \ln(P_b / Q_b) + \Sigma\, Q_b \ln(Q_b / P_b) = \Sigma\, (P_b - Q_b) \ln(P_b / Q_b)$$

Core Mathematical Properties:

- **Non-negativity:** Since $(P_b - Q_b)$ and $\ln(P_b / Q_b)$ always share the same sign, every summand is ≥ 0, ensuring PSI ≥ 0.
- **Zero Divergence:** When $P \equiv Q$, $P_b = Q_b \; \forall\, b \Rightarrow \ln(1) = 0 \Rightarrow$ PSI = 0.0.
- **Zero-Frequency Regularization ($\varepsilon = 10^{-4}$):** If any bin has 0 occurrences, P_b or Q_b is clamped to max(pct, 10^{-4}) to prevent division by zero or ln(0).

PSI Value Range	Drift Classification	Production Meaning & Recommended Action
PSI < 0.10	**none**	Distributions are essentially stable. No immediate operational intervention required.
0.10 ≤ PSI < 0.25	**moderate**	Measurable drift detected. Increase monitoring, log audit frames, schedule retraining.
PSI ≥ 0.25	**significant**	Severe distribution divergence. Trigger high-priority alerts, fallback model, or immediate retraining.

3. Implementation Logic & Algorithms (`src/drift_monitor.py`)

The core monitoring logic consists of four modular functions designed for determinism, robust edge-case handling, and zero memory leaks:

Function 1: `extract_confidence_scores(camera_dir)`

```
def extract_confidence_scores(camera_dir):
    scores = []
    pattern = os.path.join(camera_dir, '*.jpg')
    for img_path in sorted(glob.glob(pattern)):
        with Image.open(img_path) as raw_img:
            img = raw_img.convert('RGB')
            detections = det.detect(img)
            for d in detections:
                scores.append(float(d.score))
    return scores
```

Logic: Traverses camera_dir with sorted(glob.glob(...)) guaranteeing deterministic file ordering. Uses a context manager with Image.open() and convert('RGB') to prevent OS file descriptor leakage, runs the mock detector over each image, and aggregates confidence score floats into a flat list.

Function 2: `compute_psi(reference_scores, live_scores, n_bins=10)`

```
def compute_psi(reference_scores, live_scores, n_bins=N_BINS):
    def _bin_index(score):
        idx = int(score * n_bins)
        return min(max(idx, 0), n_bins - 1)
    ref_counts = [0] * n_bins; live_counts = [0] * n_bins
    for s in reference_scores: ref_counts[_bin_index(s)] += 1
    for s in live_scores: live_counts[_bin_index(s)] += 1
    ref_total = len(reference_scores) or 1; live_total = len(live_scores) or 1
    psi = 0.0
    for i in range(n_bins):
        ref_pct = max(ref_counts[i] / ref_total, 1e-4)
        live_pct = max(live_counts[i] / live_total, 1e-4)
        psi += (live_pct - ref_pct) * math.log(live_pct / ref_pct)
    return float(psi)
```

Logic: Discretizes [0.0, 1.0] into 10 bins. The boundary value 1.0 is securely mapped to bin 9 via min(..., n_bins - 1). Relative proportions are clamped at 1e-4, and symmetric divergence is accumulated with natural log.

Functions 3 & 4: `classify_drift(psi)` & `summarize_scores(scores)`

```
def classify_drift(psi):
    if psi < PSI_MODERATE_THRESHOLD: return 'none'
    elif psi < PSI_SIGNIFICANT_THRESHOLD: return 'moderate'
    else: return 'significant'

def summarize_scores(scores):
    if not scores:
        return {'count': 0, 'mean': 0.0, 'std': 0.0, 'min': 0.0, 'max': 0.0}
    count = len(scores)
    std_val = round(float(statistics.stdev(scores)), 4) if count >= 2 else 0.0
    return {'count': count, 'mean': round(float(statistics.mean(scores)), 4),
            'std': std_val, 'min': round(float(min(scores)), 4),
            'max': round(float(max(scores)), 4)}
```

Logic: classify_drift strictly respects threshold boundaries (<0.10, [0.10, 0.25), ≥0.25). summarize_scores safeguards against single-element lists (where statistics.stdev raises StatisticsError) by conditionally returning 0.0.

4. Mock Detector Mechanics (`src/mock_detector.py`)

• **Reddish Pixel Heuristic:** A pixel is classified as vehicle foreground if R > 120 and R - G > 40 and R - B > 40.
• **4-Connected Flood Fill:** An iterative DFS algorithm identifies connected blobs of reddish pixels and derives bounding box coordinates [bx, by, bw, bh].
• **Confidence Scoring:** Calculated as score = min(0.98, 0.4 + |pts| / (bw * bh + 1)), weighting fill density.
• **JPEG Compression Roundtrip:** Fixtures are serialized to JPEG and reloaded during generation to replicate disk-based compression shifts.

5. Student Fixture Generation & Pipeline Results

Student-Specific Fixture Generation (`generate_for_student.py`):
Seeded deterministically with student ID **142301024**, deriving seed **706208327**:
• **Reference Camera (`camera_A_daylight`):** Daylight background RGB (170, 175, 180), noise level 8, box scale 1.0.
• **Live Camera (`camera_B_lowlight`):** Night background RGB (60, 62, 70), noise level 22, box scale 0.7.
• **Output:** Generated 40 images (20 per camera) and 149 annotations in _annotations.coco.json.

Executing python src/run_pipeline.py extracted confidence scores from both camera streams and evaluated PSI:

Distribution Metric	Reference (camera_A_daylight)	Live (camera_B_lowlight)	Statistical Shift (Δ)
Detection Count (N)	73 detections	75 detections	+2 detections
Mean Confidence (μ)	0.9698	0.9767	+0.0069
Standard Deviation (σ)	0.0524	0.0290	-0.0234 (variance compression)
Minimum Score	0.6240	0.7290	+0.1050 (loss of low-conf tail)
Maximum Score	0.9800	0.9800	0.0000
Calculated PSI	—	**0.1346**	**Moderate Drift (0.10 ≤ PSI < 0.25)**

Physical and Statistical Interpretation:
In camera_B_lowlight, the darker ambient background and elevated per-pixel noise (noise=22) prevent faint, noisy vehicle fringes from triggering the reddish threshold. Consequently, the low-confidence tail of detections is truncated (min score rises from 0.6240 to 0.7290), and surviving detections are tight, dense vehicle centers with compressed variance (σ drops from 0.0524 to 0.0290). This structural shift across confidence bins produces a PSI of **0.1346**, triggering a **moderate drift alert** as anticipated.

Emitted `drift_report.json`:


```
{
  "reference_camera": "camera_A_daylight",
  "live_camera": "camera_B_lowlight",
  "reference_summary": {
    "count": 73, "mean": 0.9698, "std": 0.0524, "min": 0.624, "max": 0.98
  },
  "live_summary": {
    "count": 75, "mean": 0.9767, "std": 0.029, "min": 0.729, "max": 0.98
  },
  "psi": 0.1346,
  "drift_level": "moderate",
  "thresholds": {"moderate": 0.1, "significant": 0.25}
}
```

6. Observability Analysis: What Confidence Monitoring Misses

Confidence score monitoring tracks output distribution drift *P(Y_hat)* without requiring labels. However, as emphasized in lectures, unsupervised confidence monitoring has critical blind spots:

Drift Dimension	Caught by Confidence PSI?	Mechanism & Production Vulnerability
Covariate Shift *P(X) changes*	**Partially**	Catches input changes only when image degradation (e.g. darkness, noise) visibly depresses or skews the model's confidence distribution.
Concept Drift *P(Y|X) changes*	**NO (Completely Misses)**	A model can be **confidently wrong**! Under concept drift, background artifacts or novel non-vehicle entities can trigger 0.95+ confidence false positives. The confidence distribution appears healthy while actual precision crashes.
False Negatives *Silent Misses*	**NO (Completely Misses)**	If poor conditions cause vehicles to be completely missed, **no detection or confidence score is ever emitted**. Score metrics on surviving detections look normal while real-world recall collapses.
Label / Prior Drift *P(Y) changes*	**NO**	Shifts in the true proportions of vehicles (e.g. surge in commercial trucks or two-wheelers) cannot be diagnosed from confidence alone.

Roadmap: What to Monitor When Ground-Truth Labels Arrive (Delayed Verification):
1. **Supervised Detection Metrics:** Mean Average Precision (mAP@[.50:.95], mAP@50), Precision, Recall, and False Negative Rate (FNR).
2. **Class Confusion Dynamics:** Confusion matrices across the 14 vehicle classes to pinpoint misclassifications.
3. **Upstream Input Feature Telemetry:** Image luminance, contrast, sharpness (Laplacian variance), and noise variance to catch covariate shift before delayed labels arrive.

7. Test Suite Verification & Quality Assurance

All 9 unit and smoke tests in tests/test_smoke.py pass successfully:

```
tests/test_smoke.py::test_extract_confidence_scores_returns_floats_in_range PASSED [ 11%]
tests/test_smoke.py::test_extract_confidence_scores_is_deterministic PASSED         [ 22%]
tests/test_smoke.py::test_compute_psi_is_zero_for_identical_distributions PASSED   [ 33%]
tests/test_smoke.py::test_compute_psi_is_positive_for_shifted_distributions PASSED [ 44%]
tests/test_smoke.py::test_compute_psi_small_for_similar_distributions PASSED       [ 55%]
tests/test_smoke.py::test_classify_drift_bands PASSED                              [ 66%]
tests/test_smoke.py::test_summarize_scores_shape PASSED                            [ 77%]
tests/test_smoke.py::test_summarize_scores_single_value_has_zero_std PASSED       [ 88%]
tests/test_smoke.py::test_full_pipeline_runs_and_flags_camera_drift PASSED         [100%]
============================== 9 passed in 1.18s ===============================
```

In addition, src/drift_monitor.py passes flake8 linting with 0 errors or warnings.